CHAPTER 3

Working with Data

Conceptually, most software applications can be simplified to this: collect data, operate on data, and then return data to the user. This simplification avoids a whole host of complexities (user interface, database, and so on), but it holds true more often than not. Data and operations on that data are basically the entirety of an application.

In the previous chapter, you learned that functions and modules are used to organize your application. Functions do the work of your application, but they operate on data. So, the way that your data is defined has a big impact on how easy it is to write functions. Luckily, Elixir gives you all of the tools needed to create a clean data layer.

We'll explore two of the data types that we didn't cover in the previous chapter: lists and maps. You'll see the most common ways to create and interact with these data types. You'll also learn how to implement a clean data layer using a special type of map. Finally, we'll go over how you can iterate and modify your data structures.

We'll start with lists.

Lists—Not Arrays

Lists contain data and can be enumerated on. On the surface, they are similar to Ruby arrays, but there are pretty significant differences under the hood that result in totally different performance characteristics. As a result, you'll use lists differently in Elixir than you would in Ruby.

In addition to lists, there's a list alternative that throws a bit of a wrench at new Elixir programmers. You'll learn about tuples and how they differ from lists. Don't worry though, it's a pretty straightforward difference.